Sprint 1 – Week 1 Progress Report

SWENG 894 Capstone

MicroMatch

Chastidy Joanem

1. System Overview and Context

1.1 System Scope

MicroMatch is a web-based micro-internship marketplace designed to connect students with short-term, skill-based professional opportunities. The system enables organizations to post clearly scoped micro-projects and allows students to discover and engage with professional experience opportunities.

Scope of Sprint 1 (US-1 → US-5)

Sprint 1 focuses on establishing the core marketplace foundation:

- User authentication and registration
- Role-based access control (RBAC)
- Student profile management
- Organization profile management
- Micro-internship listing creation
- Basic project browsing and search

These features establish the minimum viable marketplace infrastructure required for end-to-end interaction between students and organizations.

Out of Scope for Sprint 1 (Planned for Later Sprints)

- Application workflows (US-6 → US-9)
- Deliverable submission (US-10 → US-11)
- Project completion and feedback (US-12 → US-13)
- Administrative moderation (US-14)
- Audit logging (US-15)

These features will be incrementally implemented in subsequent sprints to maintain manageable scope and iterative delivery.

1.2 System Goals and Objectives

System Goals

- Provide accessible experiential learning through micro-internships.
- Enable organizations to post and manage short-term projects.
- Support structured and secure user authentication.
- Ensure transparent and scalable architecture.
- Maintain secure role-based access control.

Alignment with Week 1 Work

Week 1 focused on establishing the technical and security foundation required to support all Sprint 1 functionality. By completing environment configuration, database schema implementation, and authentication backbone development, the system now supports secure user management and is prepared for profile and listing implementation in Weeks 2 and 3.

1.3 Mapping Between Functional Requirements and User Stories

Functional Requirement	User Story
FR-1: The system shall allow users to create an account.	US-1
FR-2: The system shall support role-based access control.	US-1
FR-3: The system shall allow students to create/edit profile.	US-2
FR-4: The system shall allow organizations to create profile.	US-3
FR-5: The system shall allow orgs to post listings.	US-4
FR-6: The system shall allow students to browse/search listings.	US-5
FR-7: The system shall allow students to apply.	US-6
FR-8: The system shall track application status.	US-8

FR-9: The system shall allow orgs to accept/reject.	US-9
FR-10: The system shall allow students to submit deliverables.	US-10
FR-11: The system shall allow orgs to review deliverables.	US-11
FR-12: The system shall allow marking projects complete.	US-12
FR-13: The system shall allow feedback submission.	US-13
FR-14: The system shall allow admin moderation.	US-14
FR-15: The system shall log system actions.	US-15

This traceability ensures alignment between formal IEEE-style functional requirements and Agile user stories used for sprint planning.

2. Sprint Planning

2.1 Groomed Product Backlog with Effort Estimation

ID	User Story	Priority	Story Points	Sprint	Rationale
US-1	Registration/Login	High	5	Sprint 1	Required for all access
US-2	Student Profile	High	5	Sprint 1	Core student data
US-3	Org Profile	High	5	Sprint 1	Required before posting
US-4	Post Listing	High	8	Sprint 1	Core marketplace feature
US-5	Browse/Search	High	5	Sprint 1	Enables discovery
US-6	Apply	High	5	Sprint 2	Core interaction
US-7	Review Applications	Medium	5	Sprint 2	Depends on applications

US-8	Track Status	Medium	3	Sprint 2	Enhances UX
US-9	Accept/Reject	Medium	3	Sprint 2	Workflow control
US-10	Submit Deliverables	Medium	5	Sprint 3	Completion flow
US-11	Review Deliverables	Medium	5	Sprint 3	Completion validation
US-12	Mark Complete	Medium	3	Sprint 3	Project closure
US-13	Feedback	Low	5	Sprint 3	Post-completion feature
US-14	Admin Moderation	Low	5	Sprint 3	Governance feature
US-15	Logging	Low	8	Sprint 3	Cross-cutting concern

Effort Estimation:

Story points follow a Fibonacci sequence, reflecting implementation complexity, dependency, and uncertainty.

2.2 Prioritization Rationale

Priority Definitions

- High: Core MVP functionality required for end-to-end workflow.
- Medium: Important workflow enhancements dependent on core features.
- Low: Administrative or cross-cutting features not required for initial MVP validation.

Sprint 1 focuses on High-priority items because they form the foundation of the platform's authentication and marketplace functionality. Without these core features, other dependent workflows cannot be implemented or tested, so completing them first ensures meaningful end-to-end functionality for the MVP.

2.3 Sprint 1 Timeline (3 Weeks)

Week 1 – Authentication Foundation

- Environment setup
- Database schema implementation

- User model creation
- Password hashing
- JWT authentication
- Role-based access logic

Week 2 – Profile & Listing Features

- Student profile CRUD
- Organization profile CRUD
- Listing creation

Week 3 – Marketplace Discovery

- Browse listings
- Basic search functionality
- End-to-end validation of Sprint 1 workflow

2.4 Sprint 1 – Week 1 Backlog

Sprint Goal (Week 1)

Establish the foundational system architecture and implement the authentication backbone required to support all subsequent marketplace features.

Sprint Item	Story Points	Related US	Reason Selected
Spike: Environment Setup	3	---	Required for development
Spike: DB Schema	5	---	Enables persistent storage
Implement User Model	3	US-1	Core authentication entity
Implement Registration Endpoint	5	US-1	Account creation
Implement JWT Authentication	5	US-1	Secure login
Implement Role Field & Validation	3	US-1	RBAC foundation

Capacity Reasoning:

Week 1 capacity was calculated based on a single-developer workload over one week (approximately 15–20 effective development hours outside of documentation and academic obligations).

The total planned workload for Week 1 was **24 story points**, distributed across:

- 8 story points for infrastructure spikes
- 16 story points for authentication implementation

Authentication was selected because it is a foundational dependency for all remaining Sprint 1 features (profiles and listings). Completing authentication in Week 1 reduces integration risk in Weeks 2 and 3.

The workload was intentionally front-loaded with security and architecture tasks to prevent downstream rework. Since authentication touches database schema, API structure, and role validation, implementing it early stabilizes system design and improves sprint predictability.

All planned story points were completed within the estimated capacity, indicating realistic estimation and effective scope control.

2.4 Definition of Done (DoD)

For Week 1 backlog items, the DoD includes:

- Development environment fully configured
- PostgreSQL schema implemented with primary/foreign keys
- User model created
- Password hashing implemented
- JWT authentication functional
- Role-based access control logic implemented
- Unit tests pass successfully
- Code committed and pushed to GitHub
- CI pipeline validates build

2.5 Acceptance Criteria (Week 1)

Spike: Environment Setup

Priority: Medium / Spike

AC-ENV-01

- Given the developer installs and configures the IDE, frontend framework, backend framework, and version control system
- When setup is completed
- Then a functional development environment is available for coding

AC-ENV-02

- Given the developer runs npm start and uvicorn main:app
- When both services start successfully
- Then the frontend and backend are accessible locally without runtime errors

Spike: Database Schema Design

Priority: Medium / Spike

AC-DB-01

- Given initial entities are defined (Users)
- When the schema is implemented in PostgreSQL
- Then tables are created with correct fields, data types, primary keys, and foreign keys

AC-DB-02

- Given sample test data is inserted
- When SQL queries are executed
- Then referential integrity constraints are enforced without errors

US-1: Registration / Login

Related FR: FR-1, FR-2

Priority: High

Registration --- Valid Credentials

AC-US1-01

- Given a new user provides a valid email, password, and role
- When the registration endpoint is called

- Then
  - A new user record is created in the database
  - The password is stored in hashed form
  - A success response is returned

Registration --- Duplicate Email

AC-US1-02

- Given a user attempts to register using an email that already exists
- When the user submits the registration form
- Then
  - The system rejects the request
  - No duplicate user record is created
  - An appropriate error response is returned

Login --- Valid Credentials

AC-US1-03

- Given a registered user with valid credentials
- When the login endpoint is called with correct email and password
- Then
  - Credentials are validated
  - A JWT authentication token is generated
  - The token is returned in the response

Login --- Invalid Credentials

AC-US1-04

- Given a registered user
- When the login endpoint is called with incorrect credentials
- Then
  - Authentication fails
  - No JWT token is issued
  - An appropriate error response is returned

Role-Based Access Control

AC-US1-05

- Given an authenticated user with an assigned role (Student, Organization, Admin)

- When the user accesses protected endpoints
- Then
  - Access is granted only to resources permitted for that role
  - Unauthorized access attempts are rejected

Test Case ID	User Story	Functional Requirement	Acceptance Criteria
TC-01	Spike: Environment Setup	N/A	AC-ENV-01
TC-02	Spike: Environment Setup	N/A	AC-ENV-02
TC-03	Spike: Database Schema Design	N/A	AC-DB-01
TC-04	Spike: Database Schema Design	N/A	AC-DB-02
TC-05	US-1	FR-1	AC-US1-01
TC-06	US-1	FR-1	AC-US1-02
TC-07	US-1	FR-1	AC-US1-03
TC-08	US-1	FR-1	AC-US1-04
TC-09	US-1	FR-2	AC-US1-05

Acceptance criteria follow the Given–When–Then convention to ensure testable, verifiable results.

3. Software Testing

3.1 Week 1 Testing Focus

- Validate development environment functionality
- Validate schema integrity and relationships
- Verify authentication endpoints
- Confirm password hashing and JWT token issuance
- Ensure role validation logic functions correctly

3.2 User Acceptance Test Specification (Partial)

Note: These are planned test cases for Week 1 and will be executed in subsequent weeks.

Test Case ID	Related Requirement	Preconditions	Steps	Expected Result
TC-01	N/A	Tools installed	Launch frontend and backend	Both services start successfully

TC-02	N/A	Services running	Access local URLs	Frontend and backend accessible
TC-03	N/A	Database running	Execute schema creation script	Tables created with correct constraints
TC-04	N/A	Sample data inserted	Execute relational queries	Foreign key constraints enforced
TC-05	FR-1	User does not exist	Call registration endpoint with valid data	User created, password hashed
TC-06	FR-1	Existing user email	Call registration endpoint	Error returned, no duplicate record
TC-07	FR-1	Registered user exists	Call login endpoint with valid credentials	JWT token returned
TC-08	FR-1	Registered user exists	Call login with invalid password	Authentication rejected
TC-09	FR-2	Authenticated user	Access protected endpoint	Access allowed/denied based on role

3.3 Requirements-to-Test Case Traceability Matrix

Functional Requirement	User Story	Test Case IDs
FR-1: User account creation & authentication	US-1	TC-05, TC-06, TC-07, TC-08
FR-2: Role-based access control	US-1	TC-09
Infrastructure (Spikes)	N/A	TC-01, TC-02, TC-03, TC-04

4. Source Code Development

4.1 Summary of Contributions

- GitHub repository initialized
- Project architecture structured (frontend/backend/database layers)
- Development environment configured (React, FastAPI, PostgreSQL)
- Database schema implemented with initial entities
- User model implemented
- Registration endpoint created
- Password hashing integrated

- JWT authentication configured
- Role-based access logic added
- Unit tests for authentication implemented
- CI pipeline configured

4.2 Repository Access and Version Control

The repository is hosted on GitHub at:
https://github.com/chazdj/MicroMatch

All sprint artifacts and commits are visible. Access permissions have been verified.

4.3 Important Commits and Relation to Sprint Items

Commit	Related To	Purpose
Initialize project structure for MicroMatch	Spike: Environment Setup	Sets up folder structure, Git, and initial project scaffolding
Setup FastAPI backend environment	Spike: Environment Setup	Configures development environment for backend
Initialize React frontend	Spike: Environment Setup	Configures development environment for frontend
Database schema implemented via Docker PostgreSQL	Spike: Database Schema	Implements initial Users table with primary keys, unique email, and role field
User SQLAlchemy model implemented.	US-1	Sets up core authentication entity with secure password storage
Registration endpoint fully implemented.	US-1	Allows new users to register accounts
Login endpoint implemented.	US-1	Implements secure login functionality
JWT validation dependency implemented.	US-1	Implements secure login functionality with token-based authentication
Implement Role-based access logic	US-1	Adds RBAC foundation for protected endpoints

4.4 Burndown Chart Analysis

Total Story Points Committed: 24
Completed: 24
Remaining: 0

Week 1 successfully delivered all planned foundational authentication and infrastructure components, positioning the system for profile and listing implementation in Week 2.

5. Architectural Design Discussion

The system follows a layered architecture:

- **Presentation Layer**
  - React frontend responsible for UI rendering and interaction.
- **Application Layer**
  - FastAPI backend responsible for authentication, business logic, and workflow enforcement.
- **Data Layer**
  - PostgreSQL database maintaining persistent data with referential integrity.

Architectural Rationale

- Separation of concerns enhances maintainability.
- JWT-based authentication enables stateless scalability.
- Role-based access control enforces security boundaries.
- Layered architecture supports incremental Agile development.

Week 1 architectural decisions ensure a stable and secure foundation for future sprint features.

6. Backlog Grooming

Sprint 1 scope was refined to focus exclusively on US-1 through US-5. Application workflows and completion features were intentionally deferred to later sprints to ensure manageable implementation scope and incremental delivery.

No story point changes were required.

7. Conclusion

Week 1 of Sprint 1 successfully established the architectural and security foundation of the MicroMatch platform. Authentication and role-based access control have been implemented, the database schema has been validated, and the development environment is fully operational.

The system is now prepared for implementation of student and organization profiles in Week 2 and marketplace listing and browsing functionality in Week 3. This structured, incremental delivery ensures alignment with MVP goals while maintaining architectural stability and scalability.